arXiv:2306.00802v2 [stat.ML] 6 Nov 2023

Birth of a Transformer: A Memory Viewpoint

Alberto Bietti[*] Flatiron Institute
Vivien Cabannes FAIR, Meta
Diane Bouchacourt FAIR, Meta
Hervé Jégou FAIR, Meta
Léon Bottou FAIR, Meta

Abstract

Large language models based on transformers have achieved great empirical successes. However, as they are deployed more widely, there is a growing need to better understand their internal mechanisms in order to make them more reliable. These models appear to store vast amounts of knowledge from their training data, and to adapt quickly to new information provided in their context or prompt. We study how transformers balance these two types of knowledge by considering a synthetic setup where tokens are generated from either global or context-specific bigram distributions. By a careful empirical analysis of the training process on a simplified two-layer transformer, we illustrate the fast learning of global bigrams and the slower development of an "induction head" mechanism for the in-context bigrams. We highlight the role of weight matrices as associative memories, provide theoretical insights on how gradients enable their learning during training, and study the role of data-distributional properties.

1 Introduction

As large language models (LLMs) are growing in usage and deployment, it is increasingly important to open the black box and understand how they work. A better understanding can help with interpretability of how these models make decisions, and will be crucial to improve these models and mitigate their failure cases, such as hallucinations or reasoning errors.

An important ingredient in the success of recent LLMs is their ability to learn and reason from information present in their context [7]. These "in-context" learning capabilities are often attributed to the transformer architecture [53], in particular its self-attention blocks, which are able to carefully select parts of the input sequence in order to infer plausible next tokens. Additionally, predictions may require "global" knowledge, such as syntactic rules or general facts, which may not appear in the context and thus needs to be stored in the model.

In order to better understand how transformers develop these capabilities during training, we introduce a synthetic dataset that exhibits both aspects. It consists of sequences generated from a bigram language model, but where some of the bigrams are specific to each sequence. Then, the model needs to rely on in-context learning for good prediction on the sequence-specific bigrams, while the global bigrams can be guessed from global statistics conditioned on the current token. While one-layer transformers fail to reliably predict the in-context bigrams, we find that two-layer transformers succeed by developing an *induction head* mechanism [17, 41], namely a "circuit" of two attention heads that allows the transformer to predict $\mathtt{b}$ from a context $[\cdots, \mathtt{a}, \mathtt{b}, \cdots, \mathtt{a}]$, and which appears to be ubiquitous in transformer language models [41, 56].

In order to obtain a fine-grained understanding of how this in-context mechanism emerges during training, we further simplify the two-layer architecture by freezing some of the layers at random initialization, including embeddings and value matrices. This focuses our study on attention and feed-forward mechanisms, while avoiding the difficulty of learning representations, which may

[*]Work done while at FAIR, Meta.

37th Conference on Neural Information Processing Systems (NeurIPS 2023).

require complex nonlinear dynamics [16, 34, 46]. This simplification also allows us to introduce a natural model for individual weight matrices as *associative memories*, which store input-output or key-value pairs of embeddings through their outer products. Random high-dimensional embeddings are particularly well-suited to this viewpoint thanks to their near-orthogonality. We provide a detailed empirical study of the training dynamics, by measuring how quickly each weight matrix learns to behave as the desired associative memory, studying how this is affected by data-distributional properties, and investigate the order in which layers are learned: the model first finds the right output associations from the current token and from uniform attention patterns, then the attention heads learn to focus on the correct key-value pairs. We then present theoretical insights on this top-down learning process through population gradient dynamics. Despite its simplicity, our setup already provides useful insights on the internal structure of transformer language models and its evolution throughout training, paving the way for a better understanding of LLMs. We hope that our insights may lead to future research and improvements for LLM practitioners, *e.g.*, for optimization algorithms, data pre-processing and selection, interpretability, fine-tuning, and model editing.

In summary, we make the following contributions:

- We introduce a new synthetic setup to study global vs in-context learning: sequences follow bigram language models, where some bigrams change across sequences and others do not.
- We view the transformer's weight matrices as associative memories that learn to store specific pairs of embeddings, and use this to derive a simplified but more interpretable model for our task.
- We empirically study the training dynamics with careful probing: global bigrams are learned first, then the induction head is formed by learning appropriate memories in a top-down fashion.
- We give theoretical insights on training dynamics, showing how a few top-down gradient steps on the population loss can recover the desired associative memories by finding signal in noisy inputs.

Related work. After the success of transformer language models for in-context learning was found [7], several works have studied how in-context learning may arise in various contexts [1, 9, 39, 44, 49, 55, 59]. Multiple recent papers have introduced synthetic tasks in order to better understand and interpret transformers [10, 34, 40, 63]. Several works have attempted to understand internal mechanisms in transformers that are responsible for certain behaviors, an area known as "mechanistic interpretability" [17, 18, 36, 40, 41, 56]. Memory and neural networks have a long history of connections [6, 19, 20, 22, 27, 28, 31, 35, 51, 57, 58]. The associative memories we consider bear similarity to [30, 58], though we use continuous input/outputs. The reader may also be interested in Fast Weight programmers [47, 48]. The use of random vectors for storing memories is related to [24]. Our approach to probing based on memory recall is related to techniques in [14, 18], though motivated differently. [15, 33, 37] study statistical and approximation properties of transformers, highlighting benefits of sparse attention patterns, but do not consider training dynamics. [26, 32, 50, 52] provide theoretical analyses of learning dynamics in transformers and other attention models, but consider different data setups and focus on single-layer architectures, while we focus on two-layer models and take a different viewpoint based on associative memories.

2 Background

This section provides background on transformer architectures and induction head mechanisms.

Transformer architecture. Transformers [53] operate on sequences of embeddings by alternating self-attention operations and token-wise feed-forward layers. We focus on decoder-only, auto-regressive architectures with a causal attention mask, which are commonly used in large language models trained for next-token prediction [7, 12, 42, 43]. We ignore normalization layers in order to simplify the architecture, since its stability benefits are not as crucial in the small models we consider. Given an input sequence of tokens $z_{1:T} \in [N]^T$ of length T, where N is the vocabulary size, the transformer operates as follows:

- **Token embeddings**: each discrete token is mapped to a d-dimensional embedding via an embedding map $W_E \in \mathbb{R}^{d \times N}$. We will denote the embeddings of tokens z_t by $x_t := w_E(z_t)$, where $w_E(j)$ is the j-th column of W_E.
- **Positional embeddings**: the positional embeddings $p_t \in \mathbb{R}^d$, $t \in [T]$, are added to each token embedding depending on its position in the sequence, leading to the following input embeddings:

$$x_t := x_t + p_t = w_E(z_t) + p_t. \tag{1}$$

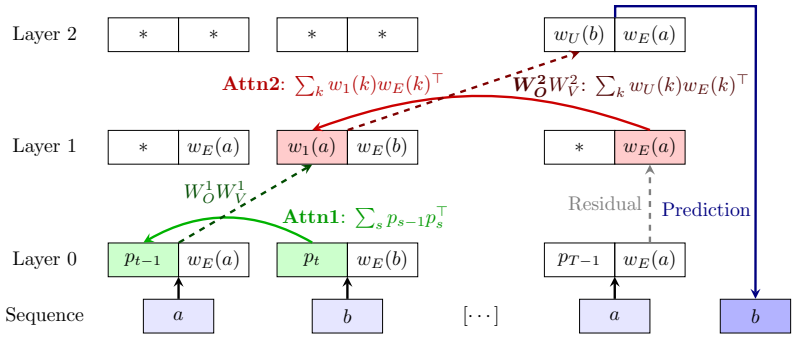

Figure 1: **Induction head mechanism**. Induction heads are a two-layer mechanism that can predict b from a context $[\ldots, a, b, \ldots, a]$. The first layer is a *previous token head*, which attends to the previous token based on positional embeddings ($p_t \to p_{t-1}$) and copies it after a remapping ($w_E(a) \to w_1(a) := W_O^1 W_V^1 w_E(a)$). The second layer is the *induction head*, which attends based on the output of the previous token head ($w_E(a) \to w_1(a)$) and outputs the attended token, remapped to output embeddings ($w_E(b) \to w_U(b)$). Boxes in the diagram represent different embeddings in superposition on each token's residual stream (we omit some irrelevant ones for clarity, *e.g.*, positional embeddings in upper layers), and attention and output associations are shown with the associative memory viewpoint presented in Section 4.

- **Attention blocks**: given an input sequence $x_{1:T} \in \mathbb{R}^{d \times T}$ of embeddings, the causal attention block computes, for $W_K, W_Q, W_V, W_O \in \mathbb{R}^{d \times d}$ (key, query, value, output), and for each t,

$$x'_t := W_O W_V x_{1:t} \sigma(x_{1:t}^\top W_K^\top W_Q x_t) \in \mathbb{R}^d, \tag{2}$$

  where σ takes the softmax of its elements, leading to an attention of the "values" $W_V x_t$ with weights proportional to $\exp((W_K x_s)^\top (W_Q x_t))$. Note that the attention operation usually considers multiple "heads" that each projects the input to a lower dimension. Here we stick to a single head for simplicity, since it will be sufficient for our purposes. Rewriting (2) on each t as $x'_{1:T} = \mathcal{A}(x_{1:T}; W_K, W_Q, W_V, W_O)$, the ℓ-th layer of the transformer applies attention with layer-specific parameters along with a residual connection as follows:[2]

$$x_{1:T} := x_{1:T} + \mathcal{A}(x_{1:T}; W_K^\ell, W_Q^\ell, W_V^\ell, W_O^\ell)$$

- **Feed-forward blocks**: feed-forward blocks operate on individual token embeddings after each attention block, typically by applying a one-hidden-layer MLP to each token, denoted $\mathcal{F}(\cdot; W_F)$, with a residual connection: at layer ℓ, we have

$$x_t := x_t + \mathcal{F}(x_t; W_F).$$

  Our simplified setup will linear feed-forward layers: $\mathcal{F}(x_t; W_F) = W_F x_t$.
- **Unembedding**: After the last transformer layer, the embeddings are mapped back to the vocabulary space $\mathbb{R}^N$ through a linear "unembedding" layer $W_U = [w_U(1), \ldots, w_U(N)]^\top \in \mathbb{R}^{N \times d}$, where we refer to the $w_U(j)$ as "output embeddings". The output of this layer is then fed into a cross-entropy loss for predicting of z_{t+1} from each x_t.

We will sometimes refer to the representations x_t for a given token t throughout layers as its *residual stream* [17], since they consist of sums of embeddings and layer outputs due to residual connections.

Induction head mechanism. Induction heads [17, 41] are a particular type of mechanism (or "circuit") in transformers that allows basic in-context prediction of the form $[\cdots, \mathtt{a}, \mathtt{b}, \cdots, \mathtt{a}] \to \mathtt{b}$. These were found to be ubiquitous in transformer language models, playing a key role in enabling various forms of in-context learning. The basic mechanism consist of two attention heads in separate layers (see Figure 1 for an illustration): (i) the first is a *previous token head* which attends to the previous token using positional information and copies its embedding to the next token; (ii) the second is the *induction head* itself, which attends using the output of the previous token head, and outputs the original token. Our work focuses on this basic copy mechanism, but we note that richer behaviors are possible, particularly when combining multiple such mechanisms (*e.g.*, [56]).

[2]We omit layer indices for simplicity of notation, and use the assignment operator $:=$ instead.